Task 1: AI-Assisted Syntax and Code Quality Review

Scenario

You join a development team and are asked to review a junior developer's Python script that fails to run correctly due to basic coding mistakes. Before deployment, the code must be corrected and standardized.

Task Description

You are given a Python script containing:

- Syntax errors
- Indentation issues
- Incorrect variable names
- Faulty function calls

```
[2]
✓ 7s
def calc_sum(a, b):
    """
    Calculates the sum of two numbers and prints the result.
    """
    result = a + b
    print("The sum is", result)
    return result

# Get user input and convert to integers
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

# Call the function with correct name and arguments
calc_sum(num1, num2)
```

```
Enter first number: 23
Enter second number: 24
The sum is 47
47
```

Task 2: Performance-Oriented Code Review

Scenario

A data processing function works correctly but is inefficient and slows down the system when large datasets are used.

Task Description

You are provided with a function that identifies duplicate values in a list using inefficient nested loops.

Using AI-assisted code review:

- Analyze the logic for performance bottlenecks
- Refactor the code for better time complexity
- Preserve the correctness of the output

```python
[3]
def find_duplicates(nums):
    seen = set()
    duplicates = set()

    for num in nums:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)

    return list(duplicates)
```

Task 3: Readability and Maintainability Refactoring

Scenario

A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description

You are given a poorly structured Python function with:

- Cryptic function names
- Poor indentation
- Unclear variable naming
- No documentation

```python
[4]
def analyze_duplicates(numbers):
    """
    Analyze a list of numbers to identify duplicates.

    Args:
        numbers (list): A list of hashable values.

    Returns:
        tuple:
            int: The number of duplicate elements found.
            list: A list of unique elements from the input.
    """
    duplicate_count = 0
    unique_values = []

    for value in numbers:
        if value not in unique_values:
            unique_values.append(value)
        else:
            duplicate_count += 1

    return duplicate_count, unique_values
```

Task 4: Secure Coding and Reliability Review

Scenario

A backend function retrieves user data from a database but has security

vulnerabilities and poor error handling, making it unsafe for production deployment.

Task Description

You are given a Python script that:

• Uses unsafe SQL query construction
• Has no input validation
• Lacks exception handling

```python
import sqlite3

def get_user_by_id(user_id):
    """
    Retrieve a user record by user ID securely.

    Args:
        user_id (int): The unique ID of the user.

    Returns:
        tuple | None: User record if found, otherwise None.
    """
    if not isinstance(user_id, int):
        raise ValueError("user_id must be an integer")

    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()

        query = "SELECT * FROM users WHERE id = ?"
        cursor.execute(query, (user_id,))

        result = cursor.fetchone()
        return result

    except sqlite3.Error as db_error:
        # Log this in real systems
        print(f"Database error: {db_error}")
        return None

    except Exception as error:
        print(f"Unexpected error: {error}")
        return None

    finally:
        if 'conn' in locals():
            conn.close()
```

Task 5: AI-Based Automated Code Review Report

Scenario

Your team uses AI tools to perform automated preliminary code reviews before human review, to improve code quality and consistency across projects.

Task Description

You are provided with a poorly written Python script.

Using AI-assisted review:

• Generate a structured code review report that evaluates:

o Code readability

o Naming conventions
o Formatting and style consistency
o Error handling
o Documentation quality
o Maintainability
The task is not just to fix the code, but to analyze and report on quality issues.

[6]
✓ 0s

```python
def analyze_duplicates(values):
    """
    Analyze a list to identify duplicate elements.

    Args:
        values (list): A list of hashable elements.

    Returns:
        tuple:
            int: Number of duplicate occurrences.
            list: List of duplicate values.
    """
    if not isinstance(values, list):
        raise TypeError("Input must be a list")

    seen = set()
    duplicates = set()
    duplicate_count = 0

    for item in values:
        if item in seen:
            duplicate_count += 1
            duplicates.add(item)
        else:
            seen.add(item)

    return duplicate_count, list(duplicates)
```